



Week 6

✓ More Example

✓ Process

- Process IDs(pid)
- A tree processes on a typical Solaris
- Process Termination (orphan, zombie)
 - Example on process termination

✓ Fork and exec()

- Exec is a family of commands
- execl, execlp
- execv



✓ Process in Java

| Content

✓ More Example

Example 1

```

int i;
for (int i = 0; i < 3; i++) {
    pid = fork();
    if (pid == 0) {
        srand(getpid());
        sleep(rand() % 5);
        printf("child i = %d\n", i);
        exit(0);
    }
    /*else {
        printf("parent i = %d\n", i);
    }*/
}
wait(NULL);
//while(wait(NULL) != -1);
printf("I am parent goodbye\n");
}

```

- This parent wait for 1 child to terminate.

```

sarun@sarun-flowx13:~/os_course/fork$ gcc -o moreforks moreforks.c
sarun@sarun-flowx13:~/os_course/fork$ ./moreforks
child i = 0
I am parent goodbye
sarun@sarun-flowx13:~/os_course/fork$ child i = 2
child i = 1

```

```

sarun@sarun-flowx13:~/os_course/fork$ ./moreforks
child i = 1
I am parent goodbye
sarun@sarun-flowx13:~/os_course/fork$ child i = 0
child i = 2

```

Example 2

```

10  int i;
11  for (int i = 0; i < 3; i++) {
12      pid = fork();
13      if (pid == 0) {
14          srand(getpid());
15          sleep(rand() % 5);
16          printf("child i = %d\n", i);
17          exit(0);
18      }
19      /*else {
20          printf("parent i = %d\n", i);
21      }*/
22  }
23  //wait(NULL);
24  while(wait(NULL) != -1);
25  printf("I am parent goodbye\n");
26  }
27

```

- wait for all its children, wait until all children terminates.

✓ Process

📌 Process IDs(pids)

Process IDs (pids)

- `pid = fork();`
- In the child: `pid == 0;`
In the parent: `pid ==` the process ID of the child.
- If `pid` is a negative value, the creation of a child process was unsuccessful.
- A program can use this `pid` difference to do different things in the parent and child. If `pid` is a negative value, the creation of a child process was unsuccessful.
- Recommended to check the error

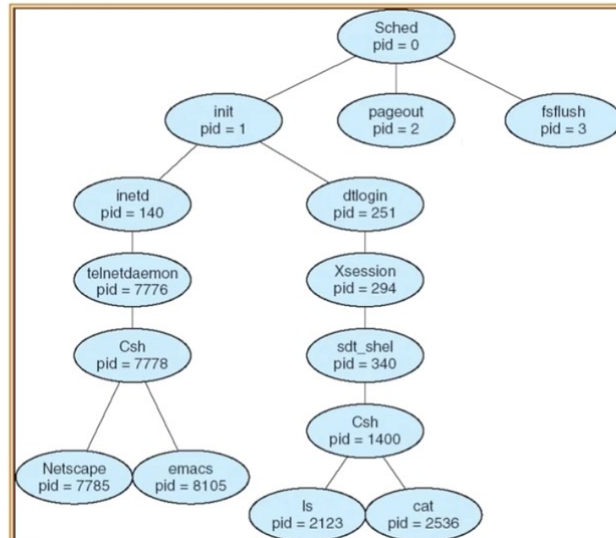
```
6  int main()
7  {
8      pid_t pid;
9      int i;
10     pid = fork();
11     if (pid > 0) { /* parent */
12         //srand(getpid());
13         //sleep(rand() % 5);
14         i = 1;
15         //sleep(10);
16         printf("I am parent i = %d\n", i);
17     }
18     else { /* child */
19         //srand(getpid());
20         //sleep(rand() % 5);
21         i = 2;
22         printf("I am child i = %d\n", i);
23     }
24     return 0;
25 }
26
```

If fork() failed → do we have output?

- Yes, the output will be in else part → 1 line of output. → wrong

A tree processes on a typical Solaris

A tree of processes on a typical Solaris



- Process can fork() any number of child process
- Child process can fork() its own child
- The only reason you can't fork → system resource are all used up.

Process Termination

Process Termination

- Process executes last statement and asks the operating system to delete it (**exit**)
 - Output data from child to parent (via **wait**)
 - Process' resources are deallocated by operating system
- Parent may terminate execution of children processes (**abort**)
 - Child has exceeded allocated resources
 - Task assigned to child is no longer required
 - If parent is exiting
 - 4 Some operating system do not allow child to continue if its parent terminates
 - All children terminated - *cascading termination*

1 way to do communication between parent and child

- Exit is use when the process need to terminate and want to send something back to its parent.

◦ wait can receive value from exit. ◦ Exit

is kind of guild line

- if the child terminate successfully → we always exit with number 0
 - if something wrong, the exit can be any number. (error code)
 - when error code get send to parent, parent can see what happened, and then can do something.
 - Parent should always wait for its children process, talk about this later.

Parent can fork its child and also kill its children

But why,

1. System has low resources → Parent may decide to kill to get resource back.
2. Task assigned to child is no longer required.
3. If parent is exiting
 - Why parent need to wait for its children process?
 - If parent doesn't wait and terminate before its children process.
 - the process will become orphan process.
 - In some OS, if it know that parent process terminate
 - 1) parent will kill all children processes, OS will kill all children of that parent.
 - 2) OS will adopt that children process.

Child process terminate before parent wait → this child process gonna be zombie process.

Zombie process will not return resource to the system. → OS must spend time to clear this zombie process later. → This is why parent should call wait statement, and make sure that wait statement is call before any child process terminate, otherwise, the process อาจจะเป็ orphan หรือ zombie

And OS has to do more task, this is why wait is important

Example on Process Termination

 Zombie (Child Terminate first)

```

3 #include <stdlib.h>
4 #include <unistd.h>
5 #include <sys/types.h>
6 int main()
7 {
8     pid_t pid;
9     int i;
10    pid = fork();
11    if (pid > 0) { /* parent */
12        //srand(getpid());
13        //sleep(rand() % 5);
14        i = 1;
15        sleep(10);
16        printf("I am parent i = %d\n", i);
17    }
18    else { /* child */
19        //srand(getpid());
20        //sleep(rand() % 5);
21        i = 2;
22        printf("I am child i = %d\n", i);
23    }
24    return 0;
25 }
26

```

slow down parent to make child terminate first.

Command line

```

sarun@sarun-flowx13: ~/os_c
sarun@sarun-flowx13:~/os_course/fork$ gcc -o forkExample forkExample.c
sarun@sarun-flowx13:~/os_course/fork$ ps
  PID TTY          TIME CMD
   72 pts/1        00:00:00 bash
 3540 pts/1        00:00:00 ps
sarun@sarun-flowx13:~/os_course/fork$ ps -l
F S  UID  PID  PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
4 S  1000   72    71  0  80   0 - 2547 do_wai pts/1    00:00:00 bash
0 R  1000 3556   72  0  80   0 - 2634 -      pts/1    00:00:00 ps
sarun@sarun-flowx13:~/os_course/fork$

```

- ps stands for process status
- with -l → show more information about process
- PID → Process ID
- PPID → Parent process id
- bash is a shell, shell is parent of ps command

```

sarun@sarun-flowx13:~/os_course/fork$ ./forkExample &
[1] 3696
sarun@sarun-flowx13:~/os_course/fork$ I am child i = 2
ps -l
F S  UID  PID  PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
4 S  1000   72    71  0  80   0 - 2547 do_wai pts/1    00:00:00 bash
0 S  1000 3696   72  0  80   0 - 590 hrtim pts/1    00:00:00 forkExample
1 Z  1000 3697 3696  0  80   0 - 0 -      pts/1    00:00:00 forkExample <defunct>
0 R  1000 3701   72  0  80   0 - 2634 -      pts/1    00:00:00 ps
sarun@sarun-flowx13:~/os_course/fork$ I am parent i = 1

```

- ./_____ & → put this to run in the background mode. → so that we can get the prompt back.

- Z stand for Zombie
- OS will clear zombie later.

Orphanage Process (parent must terminate before child)

```

3 #include <stdlib.h>
4 #include <unistd.h>
5 #include <sys/types.h>
6 int main()
7 {
8     pid_t pid;
9     int i;
10    pid = fork();
11    if (pid > 0) { /* parent */
12        //srand(getpid());
13        //sleep(rand() % 5);
14        i = 1;
15        //sleep(10);
16        printf("I am parent i = %d\n", i);
17    }
18    else { /* child */
19        //srand(getpid());
20        //sleep(rand() % 5);
21        sleep(10);
22        i = 2;
23        printf("I am child i = %d\n", i);
24    }
25    return 0;
26 }

```

delay the child to make sure parent terminate first.

```

sarun@sarun-flowx13:~/os_course/fork$ gcc -o forkExample forkExample.c
sarun@sarun-flowx13:~/os_course/fork$ ./forkExample &
[1] 3879
sarun@sarun-flowx13:~/os_course/fork$ I am parent i = 1
ps -l
F S  UID    PID  PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
4 S  1000    72   71   0  80   0 -  2676 do_wai pts/1    00:00:00 bash
1 S  1000  3880   71   0  80   0 -    590 hrtimr pts/1    00:00:00 forkExample
0 R  1000  3884   72   0  80   0 -  2634 -      pts/1    00:00:00 ps
[1]+  Done                  ./forkExample
sarun@sarun-flowx13:~/os_course/fork$ I am child i = 2

```

- forkExample have the same PPID as bash shell, what it means?
 - 71 is one of the OS processes, and now this process adopt this forkExample because the parent of this forkExample already terminates.
 - For Ubuntu, the OS will adopt the child process until it terminates

In summary, we shouldn't allow this 2 happen in our program → given more task to OS

If your program has wait statement, this 2 shouldn't have happen

Fork and exec()

- fork() and exec() sometimes always work together.

Fork and exec()

```
int main()
{
    pid_t pid;
    /* fork another process */
    pid = fork();
    if (pid < 0) { /* error occurred */
        fprintf(stderr, "Fork Failed");
        exit(-1);
    }
    else if (pid == 0) { /* child process */
        execlp("/bin/ls", "ls", NULL);
    }
    else { /* parent process */
        /* parent will wait for the child to complete */
        wait(NULL); printf("Child Complete");
        exit(0);
    }
}
```

- if you need to call to start the process of some other program → you can use exec().
- if fork() successful in above code, how many processes will be created in this program — 2 processes.
- What does the parent do?
 - wait until child terminate and print “Child Complete”
 - After parent fork a child, parent then wait here until child complete and print “Child Complete”.
- The child process then call ls command

```

C forkAndExec.c
8  pid_t pid;
9  pid = fork();
10 char *argv[] = {"ls", "-l", NULL};
11 if (pid < 0) { /* error occurred */
12     fprintf(stderr, "Fork Failed");
13     exit(-1);
14 }
15 else if (pid == 0) { /* child process */
16     execl("/bin/ls", "ls", NULL);
17     //printf("Ok\n");
18     //execvp("ls", argv);
19     //printf("I am Child\n");
20 }
21 else { /* parent process */
22     /* parent will wait for the child to complete */
23     wait (NULL);
24     printf ("Child Complete\n");
25     exit(0);
26 }
27 }
28
sarun@sarun-flowx13:~/os_course/fork$ gcc -o forkAndExec forkAndExec.c
sarun@sarun-flowx13:~/os_course/fork$ ./forkAndExec
a.out      forkAndExec      forkExample.c  moreforks.c  parent.c  waitpid.c
child      forkAndExec.c  forlExample    multiforks.c  test.txt
child.c    forkExample    moreforks      parent        waitpid
Child Complete
sarun@sarun-flowx13:~/os_course/fork$

```

- when a process called `exec()` → the code of the program that you call here replace all of your code
 - If `exec()` work successfully, now the child process will have the code of `ls` program → after this point, the code of child process and the code of parent process will be no longer the same.
 - the child process call `ls` → the code of `ls` program replace this because when it start forking, child and parent has the same code. But after the child call `ls`, the code of `ls` replace the code of a child. → child become `ls`, parent still have the same code.

Example 1

```

8   pid_t pid;
9   pid = fork();
10  char *argv[] = {"ls", "-l", NULL};
11  if (pid < 0) { /* error occurred */
12      fprintf(stderr, "Fork Failed");
13      exit(-1);
14  }
15  else if (pid == 0) { /* child process */
16      execl("/bin/ls", "ls", NULL);      I
17      printf("Ok\n");
18      //execvp("ls", argv);
19      //printf ("I am Child\n");
20  }
21  else { /* parent process */
22      /* parent will wait for the child to complete */
23      wait (NULL);
24      printf ("Child Complete\n");
25      exit(0);
26  }
27 }

```

- adding printf in line 17
- how many time the ok will be printed?

◦ 0, because ls code replace all of child process code, the ok will be replaced will ls code

```

sarun@sarun-flowx13:~/os_course/fork$ gcc -o forkAndExec forkAndExec.c
sarun@sarun-flowx13:~/os_course/fork$ ./forkAndExec
a.out      forkAndExec      forkExample.c  moreforks.c  parent.c  waitpid.c
child      forkAndExec.c  forlExample    multiforks.c  test.txt
child.c    forkExample    moreforks      parent        waitpid
Child Complete
sarun@sarun-flowx13:~/os_course/fork$

```

◦ no ok is printed.

Example 2

- use command that doesn't exist instead of ls


```

8   pid_t pid;
9   pid = fork();
10  char *argv[] = {"ls", "-l", NULL};
11  if (pid < 0) { /* error occurred */
12      fprintf(stderr, "Fork Failed");
13      exit(-1);
14  }
15  else if (pid == 0) { /* child process */
16      execl("/bin/ls", "ls", NULL);
17      printf("Ok\n");
18      //execvp("ls", argv);
19      //printf ("I am Child\n");
20  }
21  else { /* parent process */
22      /* parent will wait for the child to complete */
23      wait (NULL);
24      printf ("Child Complete\n");
25      exit(0);
26  }
27 }

```

What gonna happened?

- ok is printed, because exec() failed

```

sarun@sarun-flowx13:~/os_course/fork$ gcc -o forkAndExec forkAndExec.c
sarun@sarun-flowx13:~/os_course/fork$ ./forkAndExec
Ok
Child Complete
sarun@sarun-flowx13:~/os_course/fork$

```

More about command

parent of command is bash shell

- when you enter command, bash shell fork() a process and execute command for you.

```

sarun@sarun-flowx13:~/os_course/fork$ ps
PID TTY          TIME CMD
 72 pts/1        00:00:00 bash
11590 pts/1      00:00:00 ps
sarun@sarun-flowx13:~/os_course/fork$ ps -l
F S  UID  PID  PPID  C  PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
4 S  1000   72    71   0   80   0 - 2676 do_wai pts/1      00:00:00 bash
0 R  1000 11594   72   0   80   0 - 2634 -      pts/1      00:00:00 ps
sarun@sarun-flowx13:~/os_course/fork$

```

- exec() is used for external command only, not command built-in

```

sarun@sarun-flowx13:~/os_course/fork$ type cd
cd is a shell builtin
sarun@sarun-flowx13:~/os_course/fork$ whereis cd
cd:
sarun@sarun-flowx13:~/os_course/fork$

```

if we change to cd in a program, instead of ls → exec() failed

Exec is family of command

- manual exec()

```
EXEC(3)                                Linux Programmer's Manual                                EXEC(3)

NAME
    execl, execlp, execl, execv, execvp, execvpe - execute a file

SYNOPSIS
    #include <unistd.h>

    extern char **environ;

    int execl(const char *pathname, const char *arg, ...
              /* (char *) NULL */);
    int execlp(const char *file, const char *arg, ...
              /* (char *) NULL */);
    int execl(const char *pathname, const char *arg, ...
              /*, (char *) NULL, char *const envp[] */);
    int execv(const char *pathname, char *const argv[]);
    int execvp(const char *file, char *const argv[]);
    int execvpe(const char *file, char *const argv[],
               char *const envp[]);

    Feature Test Macro Requirements for glibc (see feature_test_macros(7)):
```

Manual page exec(3) line 1 (press h for help or q to quit)

- Exec - 2 types
 - execl
 - execv

execl(), execlp()

want to do the ls -l

```

8   pid_t pid;
9   pid = fork();
10  char *argv[] = {"ls", "-l", NULL};
11  if (pid < 0) { /* error occurred */
12      fprintf(stderr, "Fork Failed");
13      exit(-1);
14  }
15  else if (pid == 0) { /* child process */
16      execlp("/bin/ls", "ls", "-l", NULL);
17      //printf("Ok\n");
18      //execvp("ls", argv);
19      //printf ("I am Child\n");
20  }
21  else { /* parent process */
22      /* parent will wait for the child to complete */
23      wait (NULL);
24      printf ("Child Complete\n");
25      exit(0);
26  }
27 }
28

```

```

sarun@sarun-flowx13:~/os_course/fork$ gcc -o forkAndExec forkAndExec.c
sarun@sarun-flowx13:~/os_course/fork$ ./forkAndExec
total 192
-rwxr-xr-x 1 sarun sarun 17024 Aug 11 13:10 a.out
-rwxr-xr-x 1 sarun sarun 16896 Sep  1  2022 child
-rw-r--r-- 1 sarun sarun   399 Sep  1  2022 child.c
-rwxr-xr-x 1 sarun sarun 17016 Aug 11 15:08 forkAndExec
-rw-r--r-- 1 sarun sarun   608 Aug 11 15:08 forkAndExec.c
-rwxr-xr-x 1 sarun sarun 16792 Aug 11 13:36 forkExample
-rw-r--r-- 1 sarun sarun   456 Aug 11 13:36 forkExample.c
-rwxr-xr-x 1 sarun sarun 16744 Aug 25  2022 forlExample
-rwxr-xr-x 1 sarun sarun 17024 Aug 11 13:08 moreforks
-rw-r--r-- 1 sarun sarun   472 Aug 11 13:09 moreforks.c
-rw-r--r-- 1 sarun sarun   664 Aug 25  2022 multiforks.c
-rwxr-xr-x 1 sarun sarun 16872 Sep  1  2022 parent
-rw-r--r-- 1 sarun sarun   717 Sep  1  2022 parent.c
-rw-r--r-- 1 sarun sarun     5 Aug 30  2022 test.txt
-rwxr-xr-x 1 sarun sarun 16872 Sep  1  2022 waitpid
-rw-r--r-- 1 sarun sarun   771 Aug 24  2022 waitpid.c
Child Complete
sarun@sarun-flowx13:~/os_course/fork$

```

what if we change ls to random character (dsds)

```

8   pid_t pid;
9   pid = fork();
10  char *argv[] = {"ls", "-l", NULL};
11  if (pid < 0) { /* error occurred */
12      fprintf(stderr, "Fork Failed");
13      exit(-1);
14  }
15  else if (pid == 0) { /* child process */
16      execlp("/bin/ls", "dsds", "-l", NULL);
17      printf("Ok\n");
18      //execvp("ls", argv);
19      //printf ("I am Child\n");
20  }
21  else { /* parent process */
22      /* parent will wait for the child to complete */
23      wait (NULL);
24      printf ("Child Complete\n");
25      exit(0);
26  }
27 }
28

```

- it still work like ls -l

```

sarun@sarun-flowx13:~/os_course/fork$ gcc -o forkAndExec forkAndExec.c
sarun@sarun-flowx13:~/os_course/fork$ ./forkAndExec
total 192
-rwxr-xr-x 1 sarun sarun 17024 Aug 11 13:10 a.out
-rwxr-xr-x 1 sarun sarun 16896 Sep  1  2022 child
-rw-r--r-- 1 sarun sarun   399 Sep  1  2022 child.c
-rwxr-xr-x 1 sarun sarun 17016 Aug 11 15:09 forkAndExec
-rw-r--r-- 1 sarun sarun   608 Aug 11 15:08 forkAndExec.c
-rwxr-xr-x 1 sarun sarun 16792 Aug 11 13:36 forkExample
-rw-r--r-- 1 sarun sarun   456 Aug 11 13:36 forkExample.c
-rwxr-xr-x 1 sarun sarun 16744 Aug 25  2022 forlExample
-rwxr-xr-x 1 sarun sarun 17024 Aug 11 13:08 moreforks
-rw-r--r-- 1 sarun sarun   472 Aug 11 13:09 moreforks.c
-rw-r--r-- 1 sarun sarun   664 Aug 25  2022 multiforks.c
-rwxr-xr-x 1 sarun sarun 16872 Sep  1  2022 parent
-rw-r--r-- 1 sarun sarun   717 Sep  1  2022 parent.c
-rw-r--r-- 1 sarun sarun     5 Aug 30  2022 test.txt
-rwxr-xr-x 1 sarun sarun 16872 Sep  1  2022 waitpid
-rw-r--r-- 1 sarun sarun   771 Aug 24  2022 waitpid.c
Child Complete
sarun@sarun-flowx13:~/os_course/fork$

```

what if we just delete that part

```

8   pid_t pid;
9   pid = fork();
10  char *argv[] = {"ls", "-l", NULL};
11  if (pid < 0) { /* error occurred */
12      fprintf(stderr, "Fork Failed");
13      exit(-1);
14  }
15  else if (pid == 0) { /* child process */
16      execlp("/bin/ls", "-l", NULL);
17      printf("Ok\n");
18      //execvp("ls", argv);
19      //printf ("I am Child\n");
20  }
21  else { /* parent process */
22      /* parent will wait for the child to complete */
23      wait (NULL);
24      printf ("Child Complete\n");
25      exit(0);
26  }
27 }
28
sarun@sarun-flowx13:~/os_course/fork$ gcc -o forkAndExec forkAndExec.c
sarun@sarun-flowx13:~/os_course/fork$ ./forkAndExec
a.out      forkAndExec  forkExample.c  moreforks.c  parent.c  waitpid.c
child      forkAndExec.c  forlExample    multiforks.c  test.txt
child.c    forkExample    moreforks      parent        waitpid
Child Complete
sarun@sarun-flowx13:~/os_course/fork$

```

- back to c program

```

int main(int argc, char *argv[]) {

}

```

- when you run (your program name is test)
- test 1 2 3
- argc =4
- argv[0] = "test"
- argv[1] = "1"
- argv[2] = "2"
- argv[3] = "3"

```

15  else if (pid == 0) { /* child process */
16      execlp("/bin/ls", "ls", "-l", NULL);

```

- `ls` is `argv[0]` - name of the program, but it always be ignore by `exec`, but we have to have it
- `-l` is an option, `argv[1]`

what is `p` in `execlp` (`p` stands for path)

- path

```
sarun@sarun-flowx13:~/os_course/fork$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/usr/lib/wsl/lib:/mnt/c/Windows/system32:/mnt/c/Windows:/mnt/c/Windows/System32/Wbem:/mnt/c/Windows/System32/WindowsPowerShell/v1.0:/mnt/c/Windows/System32/OpenSSH:/mnt/c/Program Files (x86)/NVIDIA Corporation/PhysX/Common:/mnt/c/Program Files/NVIDIA Corporation/NVIDIA NvDLISR:/mnt/c/WINDOWS/system32:/mnt/c/WINDOWS:/mnt/c/WINDOWS/System32/Wbem:/mnt/c/WINDOWS/System32/WindowsPowerShell/v1.0:/mnt/c/WINDOWS/System32/OpenSSH:/mnt/c/Program Files/dotnet:/mnt/c/Program Files (x86)/dotnet:/mnt/c/Program Files/Git/cmd:/mnt/c/Program Files/Java/jdk-17.0.2/bin:/mnt/c/Program Files/nodejs:/mnt/c/MinGW/bin:/mnt/c/Program Files/Go/bin:/mnt/c/apache-maven-3.8.8/bin:/mnt/c/Program Files/MySQL/MySQL Server 8.0/bin:/mnt/c/Program Files/Common Files/Oracle/Java/javapath:/mnt/c/Program Files (x86)/Common Files/Oracle/Java/javapath:/mnt/c/Program Files/Racket:/mnt/c/Program Files/MySQL/MySQL Shell 8.0/bin:/mnt/c/Users/kisar/AppData/Local/Microsoft/WindowsApps:/mnt/c/Users/kisar/AppData/Local/Programs/Microsoft VS Code/bin:/mnt/c/Users/kisar/AppData/Roaming/npm:/mnt/c/Users/kisar/go/bin:/mnt/c/Users/kisar/.dotnet/tools:/snap/bin:/opt/jdk-17.0.4.1/bin
sarun@sarun-flowx13:~/os_course/fork$
```

- what does path variable?
 - path is one of environment variable in OS → so what does path variable do for us?
 - when we enter command (`ls`) → OS will search for the path by this order ▀ first, it will search in `usr/local/sbin`: is there any `ls` command or not.
- if it can't find, it go to the next path.

If we use `p`, it mean use a path value → actually in the first parameter, we can just say `ls` (not need to be a path) → and OS will search a path for us.

- what if we use `execl('ls', 'ls', '-l', NULL);`
 - error → because it can't find path for you
 - if we just using `execl`, we have to specify the full path to the command we want to execute.

Run program, start with ./

```
sarun@sarun-flowx13:~/os_course/fork$ forkAndExec
forkAndExec: command not found
sarun@sarun-flowx13:~/os_course/fork$ ./forkAndExec
```

Run program, start with ./

```
sarun@sarun-flowx13:~/os_course/fork$ forkAndExec
forkAndExec: command not found
sarun@sarun-flowx13:~/os_course/fork$ ./forkAndExec
```

- if we run without ./, it gonna be command not found, because the current folder is not in the path variable.
- put ./ to state that my program is in this current folder.

execv()

- l stands for list
- v stands for vector
 - when you use l, you must specify your argument as a list of parameter, but for v, you can declare the array and use them

```
8  pid_t pid;
9  pid = fork();
10 char *argv[] = {"ls", "-l", NULL};
11 if (pid < 0) { /* error occurred */
12     fprintf(stderr, "Fork Failed");
13     exit(-1);
14 }
15 else if (pid == 0) { /* child process */
16     //execlp("/bin/ls", "ls", "-l", NULL);
17     //printf("Ok\n");
18     execvp("ls", argv);
19     //printf ("I am Child\n");
20 }
21 else { /* parent process */
22     /* parent will wait for the child to complete */
23     wait (NULL);
24     printf ("Child Complete\n");
25     exit(0);
26 }
27 }
```

- As a SE, what situation that you might have to use this char *argv?
 - if you want to run the program and ask user to enter the option by him or herself.
- If you know exactly, the option that you want, you can use execl

Any process can call exec(), including parent, but normally parent will fork a child to call exec() → to wait for child

✓ Process in Java

- java run on jvm

Process in Java

- On most systems, the JVM is considered as an ordinary application running as a separate process on the host operating system.
- Java does not support process model, which would allow the JVM to create several processes within the same virtual machine.
- It is possible to create a process external to the JVM, however, by using the `ProcessBuilder` class, which allows a Java program to specify a process that is native to the OS.

- in 1 jvm, it doesn't support the idea of multiple process
- this means, in 1 jvm, you can run only 1 process
- 1 process per 1 jvm
- if you want to do concurrent program under java, you have to use thread.
- But even tho, java can run 1 process, it doesn't mean that you can't start the process of some other program from java program
 - you can call ls from java program
 - because java have processbuilder

Process Creation in Java

```
import java.io.*;
public class OSProcess
{
    public static void main(String[] args) throws IOException {
        if (args.length != 1) {
            System.err.println("Usage: java OSProcess <command>");
            System.exit(0);
        }
        // args[0] is the command
        ProcessBuilder pb = new ProcessBuilder(args[0]);
        Process proc = pb.start();
        // obtain the input and output streams
        InputStream is = proc.getInputStream();
        InputStreamReader isr = new InputStreamReader(is);
        BufferedReader br = new BufferedReader(isr);
        String line;
        while ( (line = br.readLine()) != null)
            System.out.println(line);
        br.close();
    }
}
```

- that you can use together to start the external command, to start the process of external command.

You can still execute other program from java program, but the major different is the process that we start by java program run under the `code OS`, not jvm → 1 jvm, 1 process only